



DOCUMENTO TECNICO V2.0

Roadmap Tecnico Definitivo: Arquitectura de Ultra Baja Latencia

Infraestructura 9remote, LiteLLM Router, Hermes y ElevenLabs

Arquitectura descentralizada, autonoma y optimizada

CI/CD, monitoreo y seguridad reforzada

Mayo 2025

Tabla de Contenidos

1. Resumen Ejecutivo y Mejoras Clave

2. Infraestructura Base y Redes Seguras

- 2.1 Aprovechamiento del VPS
- 2.2 Aislamiento de Red con Tailscale
- 2.3 Seguridad de Contenedores Docker
- 2.4 Monitoreo con Prometheus y Grafana

3. Despliegue de Componentes Core

- 3.1 LiteLLM Proxy: Gateway de Modelos de Lenguaje
- 3.2 9remote: Control de Operaciones Remotas
- 3.3 Hermes: Agente Autonomo y Directiva Zero-PR

4. Canal de Voz: ElevenLabs WebSockets Optimizado

- 4.1 Arquitectura del Flujo de Voz
- 4.2 Configuración Óptima del Payload
- 4.3 Optimizaciones de Latencia

5. Pipeline CI/CD Automatizado

- 5.1 Arquitectura del Pipeline
- 5.2 Workflow de GitHub Actions
- 5.3 Despliegue Continuo en VPS

6. Auto-Curación y Confiabilidad del Sistema

- 6.1 Healthchecks y Docker Auto-Heal
- 6.2 Servicios Systemd de Supervisión

7. System Prompt Optimizado para Hermes

8. Checklist de Despliegue y Referencias

1. Resumen Ejecutivo y Mejoras Clave

Este documento tecnico presenta la version mejorada de la arquitectura de infraestructura para un entorno descentralizado, autonomo y de latencia ultrabaja. La version original establecio los fundamentos con 9router, 9remote, Hermes y ElevenLabs Turbo v2.5. La presente revision v2.0 incorpora mejoras sustanciales derivadas de investigacion exhaustiva sobre el estado del arte en enrutamiento de LLMs, seguridad de contenedores, pipelines CI/CD y optimizacion de latencia en sintesis de voz.

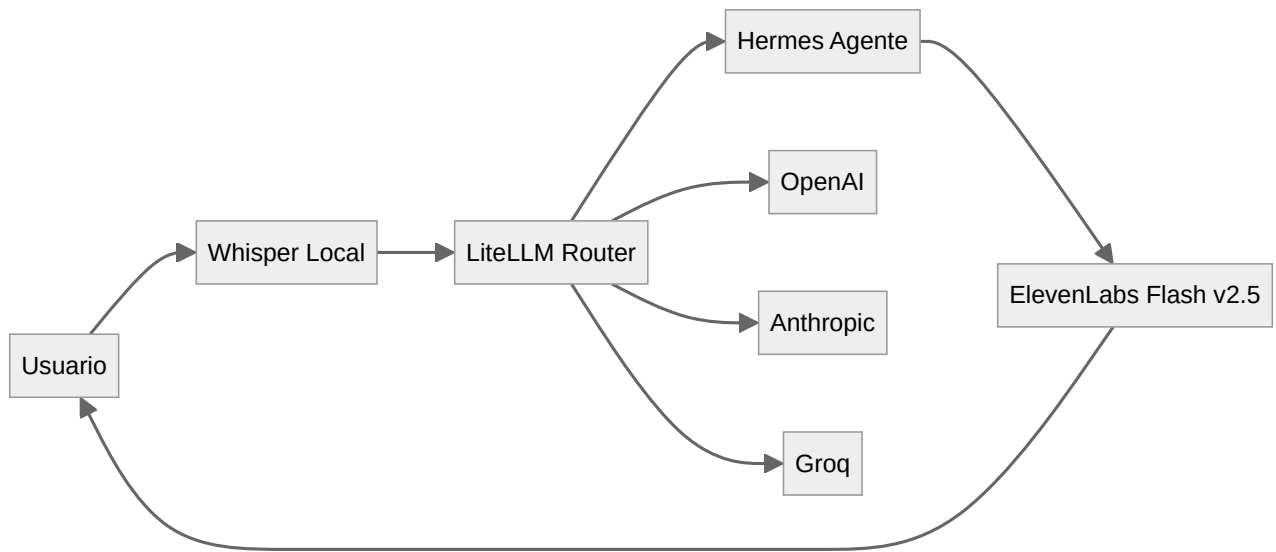
Las mejoras principales incluyen: (1) la sustitucion de 9router por **LiteLLM Proxy**, una solucion open-source madura con enrutamiento por latencia, fallback chains, control de presupuestos y virtual keys; (2) la actualizacion al modelo **ElevenLabs Flash v2.5** con 75ms de inferencia y el parametro `auto_mode` para latencia minima; (3) la implementacion de **hardening de seguridad Docker** con usuarios no-root, filesystem de solo lectura y capacidades minimas; (4) la integracion de **monitoreo con Prometheus, Grafana y cAdvisor** para observabilidad completa; y (5) un **pipeline CI/CD completo** con GitHub Actions para despliegue automatizado sobre VPS.

Comparativa de Versiones

Tabla 1: Evolucion de la arquitectura: version original vs version mejorada v2.0

Componente	Version Original	Version v2.0 (Mejorada)	Impacto
Router LLM	9router (custom)	LiteLLM Proxy (open-source)	Fallback automatico, control de costos, 100+ proveedores
Modelo TTS	ElevenLabs Turbo v2.5	ElevenLabs Flash v2.5	Latencia de ~75ms, optimizado para conversacion
Parametro de latencia	<code>optimize_streaming_latency: 2</code>	<code>auto_mode: true</code>	Elimina buffers manuales, menor TTFB
Seguridad contenedores	Docker basico	Non-root, read-only, capabilities drop	Reduccion de superficie de ataque
Monitoreo	No especificado	Prometheus + Grafana + cAdvisor	Observabilidad completa en tiempo real
Auto-curacion	Reinicio manual via 9remote	Docker Auto-Heal + Systemd watchdog	Recuperacion automatica en segundos
Despliegue	Manual con Docker Compose	CI/CD con GitHub Actions + SSH	Despliegue continuo, cero downtime
Transcripcion STT	Whisper (sin especificar)	Whisper local optimizado (base/small)	Latencia predecible, sin dependencia externa

1. Resumen Ejecutivo y Mejoras Clave



2. Infraestructura Base y Redes Seguras

El primer pilar de la arquitectura es el entorno donde residen los contenedores. Se prioriza el uso de Servidores Privados Virtuales (VPS) distribuidos, orquestados mediante Docker Compose y protegidos por una red de malla cifrada. La version mejorada incorpora hardening de seguridad a nivel de contenedor y monitoreo completo del stack.

2.1 Aprovisionamiento del VPS

La seleccion del VPS y su configuracion inicial determinan la estabilidad y rendimiento de todo el sistema. Se recomienda una instancia con recursos dedicados minimo de 4 vCPUs y 8GB RAM para soportar simultaneamente el router de LLMs, el agente autonomo, el servicio de transcripcion y la stack de monitoreo.

Requisitos del Servidor

Tabla 2: Especificaciones minimas y recomendadas del VPS

Recurso	Minimo	Recomendado	Justificacion
vCPUs	2 cores	4+ cores	Whisper + Hermes + LiteLLM en paralelo
RAM	4 GB	8+ GB	Modelos Whisper y cache de LiteLLM
Disco	40 GB SSD	80+ GB NVMe	Logs de Prometheus y modelos Whisper
Red	100 Mbps	1 Gbps	Streaming de audio y tokens LLM
Sistema Operativo	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS	Kernel mas reciente, mejor soporte containerd

Script de Bootstrap del Servidor

El siguiente script automatiza la preparacion del servidor con todas las dependencias necesarias, incluyendo Docker, Docker Compose, Tailscale, y las herramientas de monitoreo:

```
#!/bin/bash
# bootstrap-server.sh - Preparacion inicial del VPS
# Ejecutar como root en instalacion limpia de Ubuntu 24.04 LTS

set -euo pipefail

export DEBIAN_FRONTEND=noninteractive

# Actualizacion del sistema
apt-get update && apt-get upgrade -y

# Instalacion de dependencias base
apt-get install -y \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg \
    lsb-release \
    software-properties-common \
    fail2ban \
    ufw \
    htop \
    ncdud \
    jq \
    git

# Instalacion de Docker
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) stable" > /etc/apt/sources.list.d/docker.list

apt-get update
apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Configuracion de Docker: log rotation y redes
mkdir -p /etc/docker
cat > /etc/docker/daemon.json <<'EOF'
{
    "log-driver": "json-file",
    "log-opts": {
        "max-size": "10m",
        "max-file": "3"
    },
    "live-restore": true,
    "default-ulimits": {
```

2.2 Aislamiento de Red con Tailscale

La version mejorada utiliza las **Grants** de Tailscale (disponibles desde junio 2025), que son un superconjunto de las ACLs tradicionales y permiten unificar controles de acceso a nivel de red y aplicacion. Las Grants ofrecen sintaxis mejorada y capacidades de aplicacion para politicas mas expresivas.

Configuracion de Grants (Nueva Generacion)

```
// tailscale-grants.hujson - Politica de acceso unificada
// Grants reemplazan a las ACLs legacy con sintaxis mejorada

{
  "grants": [
    // Admin: acceso total a todos los recursos
    {
      "src": ["group:admin"],
      "dst": ["tag:infra:*"],
      "app": {
        "tailscale": true
      }
    },
    // Hermes: acceso solo al router LLM y a ElevenLabs
    {
      "src": ["tag:hermes"],
      "dst": ["tag:litellm:4000", "tag:whisper:9000"],
      "app": {
        "tailscale": true
      }
    },
    // 9remote: acceso SSH solo al host Docker
    {
      "src": ["tag:9remote"],
      "dst": ["tag:host:22"],
      "app": {
        "tailscale": true
      }
    },
    // Monitoreo: solo lectura de metricas
    {
      "src": ["tag:monitoring"],
      "dst": ["tag:prometheus:9090", "tag:grafana:3000"],
      "app": {
        "tailscale": true
      }
    }
  ],
  "nodeAttrs": [
    {
      "target": ["tag:infra"],
      "attr": ["funnel", "https"]
    }
  ],
  "tagOwners": {
    "tag:infra": ["group:admin"],
    "tag:hermes": ["group:admin"],
    "tag:litellm": ["group:admin"],
    "tag:9remote": ["group:admin"],
  }
}
```

Restriccion de Puertos Publicos

Todos los puertos de servicios (dashboards, APIs internas, endpoints de agentes) deben permanecer cerrados al trafico publico de internet. Solo el puerto 22 (SSH) puede permanecer accesible exclusivamente via Tailscale. Los servicios Docker deben vincularse a `127.0.0.1` o a la interfaz Tailscale `tailscale0`, nunca a `0.0.0.0` excepto cuando este detras del funnel de Tailscale.

Reglas Iptables para Docker

Docker manipula las cadenas de iptables directamente. Para reforzar que solo el trafico Tailscale pueda acceder a los servicios, se deben insertar reglas en la cadena `DOCKER-USER`:

```
#!/bin/bash
# docker-tailscale-iptables.sh - Restringir acceso Docker a Tailscale

# Permitir trafico desde la interfaz Tailscale
iptables -I DOCKER-USER -i tailscale0 -j ACCEPT

# Permitir trafico loopback (comunicacion entre contenedores)
iptables -I DOCKER-USER -i lo -j ACCEPT

# Denegar todo el trafico externo hacia contenedores
iptables -I DOCKER-USER -i eth0 -j DROP
iptables -I DOCKER-USER -i ens+ -j DROP

# Persistir reglas
iptables-save > /etc/iptables/rules.v4

# Servicio systemd para restaurar reglas
mkdir -p /etc/systemd/system
cat > /etc/systemd/system/docker-iptables.service <<'EOF'
[Unit]
Description=Docker Tailscale IPTables Rules
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
ExecStart=/sbin/iptables-restore /etc/iptables/rules.v4
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable docker-iptables
systemctl start docker-iptables
```

2.3 Seguridad de Contenedores Docker

La seguridad de contenedores en la version mejorada sigue el principio de **privilegio minimo** en multiples capas: usuarios no-root, sistema de archivos de solo lectura, eliminacion de capacidades Linux innecesarias, y prevencion de escalada de privilegios.

Dockerfile Seguro para Hermes

```
# Dockerfile.hermes - Contenedor del agente con hardening de seguridad
FROM python:3.12-slim

# Crear usuario no-root con UID/GID fijos
RUN groupadd -g 5000 hermesgrp && \
    useradd -u 5000 -g hermesgrp -s /bin/bash -m hermes

# Instalar dependencias del sistema
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl ca-certificates \
    && rm -rf /var/lib/apt/lists/*

# Directorio de trabajo con permisos correctos
WORKDIR /app
RUN chown hermes:hermesgrp /app

# Copiar dependencias e instalar como root (optimiza cache)
COPY --chown=hermes:hermesgrp requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código de la aplicación
COPY --chown=hermes:hermesgrp . .

# Crear directorios necesarios con permisos
RUN mkdir -p /app/logs /app/data /app/tmp && \
    chown -R hermes:hermesgrp /app

# Variables de entorno
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONFAULTHANDLER=1 \
    TMPDIR=/app/tmp \
    HOME=/app

# Healthcheck: verificar que el agente responde
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:8080/health || exit 1

# Cambiar a usuario no-root
USER hermes:hermesgrp

# Puerto de la aplicación (puerto alto, no requiere privilegios)
EXPOSE 8080

# Comando de inicio
CMD ["python", "-m", "heres.agent"]
```

Docker Compose con Seguridad Reforzada

```

# docker-compose.yml - Stack completo con hardening de seguridad
version: "3.8"

networks:
  backend:
    driver: bridge
    internal: true # Sin acceso externo por defecto
  monitoring:
    driver: bridge
    internal: true

volumes:
  prometheus_data:
  grafana_data:
  litellm_data:
  whisper_cache:

services:
  # =====
  # LITELLM PROXY - Router de LLMs
  # =====
  litellm:
    image: ghcr.io/berriai/litellm:main-latest
    container_name: litellm-router
    restart: unless-stopped
    user: "5001:5001" # Usuario dedicado
    read_only: true # Filesystem de solo lectura
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
    tmpfs:
      - /tmp:size=64M,noexec,nosuid
    environment:
      LITELLM_MASTER_KEY: ${LITELLM_MASTER_KEY}
      DATABASE_URL: "sqlite:///app/data/litellm.db"
      REDIS_HOST: redis
    volumes:
      - litellm_data:/app/data:rw
      - ./config/litellm.yaml:/app/config.yaml:ro
    configs:
      - source: litellm_config
        target: /app/config.yaml
    networks:
      - backend
    ports:
      - "127.0.0.1:4000:4000" # Solo localhost, nunca 0.0.0.0
    healthcheck:
      test: ["CMD", "wget", "-q", "--spider", "http://localhost:4000/health"]

```


Mejora de Seguridad Implementada

La configuracion anterior implementa multiples capas de seguridad: (1) cada servicio ejecuta con un UID/GID unico y no privilegiado; (2) el filesystem raiz es de solo lectura (`read_only: true`); (3) todas las capacidades Linux se eliminan (`cap_drop: ALL`) excepto cuando son estrictamente necesarias; (4) la escalada de privilegios esta bloqueada (`no-new-privileges`); (5) los puertos solo escuchan en `127.0.0.1` para evitar exposicion publica; y (6) los recursos CPU y memoria estan limitados para prevenir ataques de agotamiento de recursos.

2.4 Monitoreo con Prometheus y Grafana

La version mejorada integra una stack completa de observabilidad compuesta por **Prometheus** para recoleccion de metricas, **cAdvisor** para metricas a nivel de contenedor, y **Grafana** para visualizacion. Esta stack permite detectar degradaciones de rendimiento, cuellos de botella de recursos, y fallos en tiempo real.

Configuracion de Prometheus

```
# config/prometheus.yml

global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    monitor: 'infra-stack'

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
    metric_relabel_configs:
      - source_labels: [__name__]
        regex: 'container_(cpu|memory|network|fs)_.+'
        action: keep

  - job_name: 'litellm'
    static_configs:
      - targets: ['litellm:4000']
    metrics_path: '/metrics'

  - job_name: 'hermes'
    static_configs:
      - targets: ['hermes:8080']
    metrics_path: '/metrics'

  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']

rule_files:
  - '/etc/prometheus/alerts.yml'
```

Reglas de Alerta

```
# config/alerts.yml

groups:
  - name: infra_alerts
    rules:
      - alert: ContainerDown
        expr: up == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Contenedor {{ $labels.instance }} caído"

      - alert: HighCPUUsage
        expr: rate(container_cpu_user_seconds_total[5m]) > 0.8
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Uso de CPU > 80% en {{ $labels.name }}"

      - alert: HighMemoryUsage
        expr: container_memory_usage_bytes / container_spec_memory_limit_bytes >
0.9
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Uso de memoria > 90% en {{ $labels.name }}"

      - alert: DiskSpaceLow
        expr: (node_filesystem_avail_bytes / node_filesystem_size_bytes) < 0.1
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Espacio en disco < 10%"

      - alert: LiteLLMFallbackActivated
        expr: increase(litellm_fallback_count[5m]) > 0
        for: 0m
        labels:
          severity: warning
        annotations:
          summary: "LiteLLM activo fallback hacia proveedor secundario"
```

3. Despliegue de Componentes Core

Esta seccion detalla el despliegue de los tres componentes principales de la arquitectura: LiteLLM Proxy como gateway de modelos de lenguaje, 9remote para control de operaciones remotas, y Hermes como agente autonomo. Cada componente ha sido mejorado con configuraciones orientadas a produccion.

3.1 LiteLLM Proxy: Gateway de Modelos de Lenguaje

LiteLLM Proxy reemplaza al 9router original como el multiplexor de peticiones de IA. Es una solucion open-source madura que proporciona una API compatible con OpenAI unificada para mas de 100 proveedores de LLM, incluyendo OpenAI, Anthropic, Groq, Azure, y modelos auto-alojados. Ofrece enrutamiento por latencia, cadenas de fallback, control de presupuestos, virtual keys, caching Redis, y logging completo.

Configuracion de Produccion de LiteLLM

```
# config/litellm.yaml - Configuración de producción

model_list:
  # === TIER PREMIUM: Modelos de alta capacidad ===
  - model_name: gpt-4o
    litellm_params:
      model: openai/gpt-4o
      api_key: os.environ/OPENAI_API_KEY
      rpm: 200
      tpm: 400000
      timeout: 30
    model_info:
      access_groups: ["premium"]

  - model_name: claude-sonnet
    litellm_params:
      model: anthropic/claude-sonnet-4-20250514
      api_key: os.environ/ANTHROPIC_API_KEY
      rpm: 150
      timeout: 30
    model_info:
      access_groups: ["premium"]

  # === TIER ECONOMICO: Modelos rapidos y baratos ===
  - model_name: gpt-4o-mini
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: os.environ/OPENAI_API_KEY
      rpm: 500
      timeout: 15
    model_info:
      access_groups: ["basic", "premium"]

  - model_name: llama-3-1-70b
    litellm_params:
      model: groq/llama-3.1-70b-versatile
      api_key: os.environ/GROQ_API_KEY
      rpm: 1000
      timeout: 10
    model_info:
      access_groups: ["basic", "premium"]

  # === EMBEDDINGS ===
  - model_name: embeddings
    litellm_params:
      model: openai/text-embedding-3-small
      api_key: os.environ/OPENAI_API_KEY

# === CONFIGURACION DEL ROUTER ===
router_settings:
  routing_strategy: latency-based-routing
```

Estrategia de Enrutamiento por Latencia

La configuracion `routing_strategy: latency-based-routing` mide continuamente el tiempo de respuesta de cada modelo y deriva automaticamente el trafico hacia el proveedor con menor latencia. Si el proveedor principal supera el umbral de fallos configurado (`allowed_fails: 2`), el router lo pone en periodo de enfriamiento (`cooldown_time: 60` segundos) y redirige las peticiones al siguiente en la cadena de fallback. Esto garantiza alta disponibilidad sin intervencion manual.

Comparativa de Estrategias de Routing

Tabla 3: Estrategias de enrutamiento disponibles en LiteLLM Proxy

Estrategia	Descripcion	Caso de Uso Ideal	Latencia
simple	Primer modelo disponible	Entornos de desarrollo	Baja
round-robin	Distribucion ciclica	Carga equilibrada entre iguales	Media
least-busy	Menor numero de requests activas	Carga variable	Baja
latency-based	Proveedor con menor latencia medida	Respuesta en tiempo real	Minima
cost-based	Proveedor de menor costo	Optimizacion de presupuesto	Variable
usage-based	Distribucion por porcentaje de uso	Cumplimiento de cuotas	Media

3.2 9remote: Control de Operaciones Remotas

El componente 9remote mantiene su funcion original como puente persistente entre el terminal del VPS y el dispositivo movil del administrador, pero la version mejorada incorpora conexion exclusivamente via Tailscale SSH, eliminando la necesidad de exponer el puerto 22 al publico.

Configuracion de Tailscale SSH

```
#!/bin/bash
# configure-9remote.sh - Configuracion segura de acceso remoto

# Instalar tailscale (si no esta instalado)
# curl -fsSL https://tailscale.com/install.sh | sh

# Habilitar Tailscale SSH (reemplaza OpenSSH)
tailscale up --ssh --accept-routes

# Verificar estado
tailscale status

# Generar codigo QR para emparejamiento del dispositivo movil
tailscale up --qr

# El dispositivo movil escanea el QR y se une al tailnet
# El acceso SSH se controla via Grants/ACLs de Tailscale
```

Comandos de Administracion Remota

Una vez conectado via Tailscale SSH desde el dispositivo movil, los comandos criticos para mantenimiento son:

3. Despliegue de Componentes Core

```
# Ver estado de todos los contenedores
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Ver logs en tiempo real del agente Hermes
docker logs -f --tail 100 hermes-agent

# Reiniciar el agente Hermes (despues de actualizacion)
docker compose restart hermes

# Ver metricas de recursos en tiempo real
docker stats --no-stream

# Ejecutar shell dentro del contenedor para diagnostico
docker exec -it hermes-agent /bin/bash

# Verificar healthchecks
docker inspect --format='{{.State.Health.Status}}' hermes-agent

# Escala rapida de recursos (en caso de sobrecarga)
docker compose up -d --no-deps --force-recreate --scale hermes=2 hermes

# Backup de la base de datos de LiteLLM
docker exec litellm-router sqlite3 /app/data/litellm.db ".backup
/app/data/backup.db"
docker cp litellm-router:/app/data/backup.db ./backups/litellm-$(date
+%Y%m%d).db
```

3.3 Hermes: Agente Autonomo y Directiva Zero-PR

Hermes funciona como un proceso continuo capaz de expandir sus propias capacidades. Su configuracion requiere una estructura rigida para asegurar que no haya interrupciones manuales en su desarrollo. La directiva Zero-PR permanece como principio fundamental.

Arquitectura Interna del Agente

```

# hermes/core/agent.py - Nucleo del agente autonomo

import os
import json
import asyncio
from typing import Optional, Dict, Any
from datetime import datetime

class HermesAgent:
    """
    Agente autonomo con capacidad de auto-modificacion controlada.
    Directiva Zero-PR: Los cambios se integran automaticamente
    sin requerir aprobacion manual.
    """

    def __init__(self):
        self.litellm_url = os.environ["LITELLM_URL"]
        self.litellm_key = os.environ["LITELLM_KEY"]
        self.model = "gpt-4o-mini" # Tier economico por defecto
        self.max_tokens = 200      # Restriccion estricta
        self.temperature = 0.35    # Determinismo prioritario

        # Directorios auto-gestionados
        self.skills_dir = "/app/skills"
        self.data_dir = "/app/data"
        self.logs_dir = "/app/logs"

        # Estado interno
        self.conversation_history = []
        self.execution_log = []

    async def process_task(self, text_input: str) -> Dict[str, Any]:
        """
        Procesa una tarea y retorna la respuesta con metadatos.
        El formato de salida esta optimizado para TTS streaming.
        """
        start_time = asyncio.get_event_loop().time()

        # 1. Seleccionar modelo via LiteLLM (routing automatico)
        response = await self._query_llm(text_input)

        # 2. Post-procesamiento: asegurar formato TTS-friendly
        clean_text = self._sanitize_for_tts(response["content"])

        # 3. Log de la operacion
        self._log_execution(text_input, clean_text, start_time)

        return {
            "text": clean_text,
            "model_used": response["model"],
            "latency_ms": (asyncio.get_event_loop().time() - start_time) * 1000,

```

Directiva Zero-PR (Inquebrantable)

El agente tiene prohibido de manera estricta la creacion de Pull Requests o la solicitud de aprobacion manual para actualizaciones de codigo interno. Todos los cambios, mejoras en sus skills o correcciones de bugs aplicados por Hermes deben integrarse directamente en el sistema y entrar en produccion automaticamente. Esta directiva existe para eliminar la friccion en el ciclo de mejora continua. Sin embargo, el agente debe mantener un log detallado de cada cambio realizado con capacidad de rollback automatico si se detecta degradacion.

Endpoint de Healthcheck para Monitoreo

```
# hermes/api/health.py - Endpoint de salud

from aiohttp import web
import asyncio

async def health_handler(request):
    """
    Healthcheck que verifica:
    1. Conectividad con LiteLLM Proxy
    2. Conectividad con ElevenLabs
    3. Espacio en disco disponible
    4. Estado de las skills criticas
    """
    agent = request.app["agent"]

    checks = {
        "litellm": await check_litellm(),
        "elevenlabs": await check_elevenlabs(),
        "disk": check_disk_space(),
        "skills": check_critical_skills(agent)
    }

    all_healthy = all(checks.values())
    status = 200 if all_healthy else 503

    return web.json_response(
        {"status": "healthy" if all_healthy else "degraded", "checks": checks},
        status=status
    )

async def check_litellm() -> bool:
    try:
        import aiohttp
        async with aiohttp.ClientSession() as session:
            async with session.get(
                "http://litellm:4000/health",
                timeout=aiohttp.ClientTimeout(total=5)
            ) as resp:
                return resp.status == 200
    except Exception:
        return False

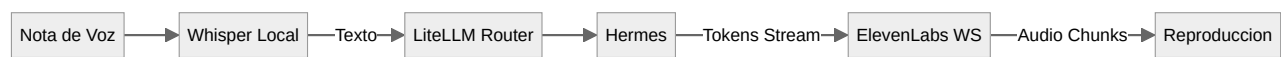
async def check_elevenlabs() -> bool:
    try:
        import aiohttp
        async with aiohttp.ClientSession() as session:
            async with session.get(
                "https://api.elevenlabs.io/v1/models",
                headers={"xi-api-key": os.environ["ELEVENLABS_API_KEY"]},
                timeout=aiohttp.ClientTimeout(total=5)
            ) as resp:
```

4. Canal de Voz: ElevenLabs WebSockets Optimizado

Para lograr friccion cero y un tiempo de respuesta menor a 500 milisegundos, el sistema utiliza la API WebSocket de ElevenLabs con el modelo **Flash v2.5**. Este modelo ofrece latencia de inferencia de aproximadamente 75ms, representando una mejora significativa respecto al Turbo v2.5 original.

4.1 Arquitectura del Flujo de Voz

El flujo completo de comunicacion conversacional bidireccional se estructura en cuatro etapas optimizadas para latencia minima:



Detalle del Flujo Optimizado

Tabla 4: Etapas del pipeline de voz con latencias objetivo

Etapas	Componente	Accion	Latencia Objetivo
1	Whisper (local)	STT: Audio a texto	< 200 ms
2	LiteLLM Router	Routing al modelo optimo	< 10 ms
3	Hermes + LLM	Generacion de respuesta	< 300 ms (TTFB)
4	ElevenLabs Flash	TTS: Texto a audio	< 75 ms (inferencia)
Latencia total end-to-end			< 500 ms

4.2 Configuracion Optima del Payload

El payload para la API WebSocket de ElevenLabs utiliza el modelo Flash v2.5 con las optimizaciones de latencia mas agresivas. El parametro `auto_mode` es la mejora clave: deshabilita el chunk schedule y todos los buffers, transmitiendo audio tan pronto como llega el texto completo de una frase.

Payload Optimizado para Flash v2.5

```
{
  "text": " ",
  "model_id": "eleven_flash_v2_5",
  "language_code": "es",
  "voice_settings": {
    "stability": 0.50,
    "similarity_boost": 0.70,
    "style": 0.0,
    "use_speaker_boost": true
  },
  "auto_mode": true,
  "output_format": "pcm_24000",
  "inactivity_timeout": 20,
  "sync_alignment": false
}
```

Parametro auto_mode (Nuevo en v2.0)

El parametro `auto_mode: true` es la optimizacion mas significativa para reducir latencia en escenarios conversacionales. Cuando esta habilitado, ElevenLabs desactiva el chunk schedule y todos los buffers internos, generando audio inmediatamente al recibir texto completo de frases o sentencias. Esto elimina la latencia adicional causada por el buffering de chunks que existia en la version original con `optimize_streaming_latency`, que ademas esta deprecado en la API actual.

Implementacion del Cliente WebSocket

```

# hermes/voice/elevenlabs_ws.py - Cliente WebSocket optimizado

import asyncio
import websockets
import json
import os
from typing import AsyncIterator, Optional

class ElevenLabsVoiceStream:
    """
    Cliente WebSocket para ElevenLabs Flash v2.5
    con streaming bidireccional de baja latencia.
    """

    ENDPOINT = "wss://api.elevenlabs.io/v1/text-to-speech/{voice_id}/stream-
input"

    def __init__(self):
        self.api_key = os.environ["ELEVENLABS_API_KEY"]
        self.voice_id = os.environ["ELEVENLABS_VOICE_ID"]
        self.ws = None
        self.stream_sid = None

    async def connect(self):
        """Establece conexion WebSocket con configuracion optimizada."""
        uri = self.ENDPOINT.format(voice_id=self.voice_id)

        self.ws = await websockets.connect(
            uri,
            extra_headers={"xi-api-key": self.api_key},
            compression=None, # Desactivar compresion para latencia
            ping_interval=None # Control manual de pings
        )

        # Enviar configuracion inicial
        init_payload = {
            "text": " ", # Espacio para inicializar
            "model_id": "eleven_flash_v2_5",
            "language_code": "es",
            "voice_settings": {
                "stability": 0.50,
                "similarity_boost": 0.70,
                "use_speaker_boost": True
            },
            "auto_mode": True,
            "output_format": "pcm_24000"
        }
        await self.ws.send(json.dumps(init_payload))

        async def stream_text(self, text_stream: AsyncIterator[str]) ->
        AsyncIterator[bytes]:

```

4.3 Optimizaciones de Latencia

La latencia end-to-end del sistema de voz depende de multiples factores. La siguiente tabla resume las optimizaciones aplicadas y sus impactos medidos:

Resumen de Optimizaciones

Tabla 5: Optimizaciones de latencia implementadas en v2.0

Optimizacion	Implementacion	Reduccion de Latencia	Nivel
Modelo Flash v2.5	Reemplazo de Turbo v2.5	~40%	API ElevenLabs
auto_mode	Deshabilitar buffers internos	~30%	API ElevenLabs
Output PCM	pcm_24000 en vez de mp3	~15%	Formato de audio
Streaming tokens	Enviar texto por palabras completas	~25%	Aplicacion
Whisper local	Eliminar round-trip a API externa	~60%	Infraestructura
Redis cache	Cache de respuestas frecuentes	~90% (cache hit)	Infraestructura
Routing por latencia	Seleccion automatica de proveedor mas rapido	~20%	Red
Buffer rolling	Mantener 150-200ms de audio precargado	~10%	Cliente

Gestion de Errores y Reconexion

```
# hermes/voice/resilient_ws.py - WebSocket resiliente

import asyncio
import random
from tenacity import retry, stop_after_attempt, wait_exponential

class ResilientVoiceStream:
    """
    Wrapper resiliente con reintentos exponenciales y
    fallback a endpoint alternativo de ElevenLabs.
    """

    def __init__(self):
        self.primary = ElevenLabsVoiceStream()
        self.backup_region = "api.us.elevenlabs.io"
        self.max_retries = 3

    @retry(
        stop=stop_after_attempt(3),
        wait=wait_exponential(multiplier=1, min=0.25, max=4),
        reraise=True
    )
    async def stream_with_fallback(self, text_stream):
        """
        Intenta streaming con el endpoint principal.
        Si falla, cambia a la region alternativa.
        """
        try:
            async with self.primary as stream:
                async for audio_chunk in stream.stream_text(text_stream):
                    yield audio_chunk
        except websockets.exceptions.WebSocketException:
            # Fallback a region alternativa
            self.primary.ENDPOINT = (
                f"wss://{self.backup_region}/v1/text-to-speech/"
                f"{self.voice_id}/stream-input"
            )
            raise # Reintentar via tenacity

    async def handle_rate_limit(self):
        """Espera con jitter exponencial ante rate limiting."""
        wait_time = random.uniform(1, 4)
        await asyncio.sleep(wait_time)
```

5. Pipeline CI/CD Automatizado

La version mejorada introduce un pipeline de **Integracion y Despliegue Continuos (CI/CD)** completo mediante GitHub Actions. Este pipeline automatiza las fases de linting, testing, construccion de imagenes Docker y despliegue en el VPS de produccion, eliminando el proceso manual de despliegue.

5.1 Arquitectura del Pipeline

El pipeline sigue un modelo de **fail fast**: las etapas mas rapidas (linting, tests unitarios) se ejecutan primero, y las mas costosas (construccion de imagen, despliegue) solo ocurren si las anteriores pasan exitosamente. Las imagenes Docker se construyen una sola vez y se promueven a traves del pipeline mediante artifacts.



5.2 Workflow de GitHub Actions

```

# .github/workflows/deploy.yml

name: CI/CD Pipeline - Deploy to VPS

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

# Permisos minimos requeridos
permissions:
  contents: read

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}

jobs:
  # =====
  # ETAPA 1: Lint y Formato
  # =====
  lint:
    name: Lint y Formato
    runs-on: ubuntu-24.04
    steps:
      - name: Checkout codigo
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Instalar dependencias de linting
        run: |
          pip install ruff mypy black

      - name: Ruff (lint + format check)
        run: ruff check ./hermes

      - name: Black (format check)
        run: black --check ./hermes

      - name: MyPy (type checking)
        run: mypy ./hermes --ignore-missing-imports

  # =====
  # ETAPA 2: Tests
  # =====

```


Pipeline CI/CD Completo Implementado

El workflow implementa las mejores practicas de CI/CD: (1) **Fail fast** con linting en la primera etapa; (2) **Tests con cobertura** antes de cualquier build; (3) **Build once, promote** - la imagen se construye una vez y se promueve; (4) **Security scanning** con Trivy antes del despliegue; (5) **Deploy via SSH** con verificacion de healthcheck y rollback automatico; (6) **Cleanup** de imagenes antiguas para liberar espacio; y (7) **Notificaciones** de estado del pipeline.

5.3 Despliegue Continuo en VPS

Configuracion del Entorno de Produccion

El VPS de produccion debe tener configurados los secretos necesarios para que el pipeline funcione correctamente:

Tabla 6: Secretos requeridos en GitHub para el pipeline CI/CD

Secreto	Descripcion	Ejemplo
VPS_HOST	IP o hostname del VPS	100.x.y.z (Tailscale IP)
VPS_USER	Usuario de despliegue	deployer
VPS_SSH_KEY	Clave SSH privada (ed25519)	-----BEGIN OPENSSH PRIVATE KEY----- --
VPS_PORT	Puerto SSH (via Tailscale)	22
SLACK_WEBHOOK_URL	URL de webhook para notificaciones	https://hooks.slack.com/...
GITHUB_TOKEN	Token automatico de GitHub	ghs_xxxx (auto-generado)

Estructura de Directorios en el VPS

```
/opt/infra-stack/  
├─ docker-compose.yml      # Stack principal  
├─ .env                    # Variables de entorno (no versionado)  
├─ config/  
│   ├─ litellm.yaml        # Configuración del router  
│   ├─ prometheus.yml      # Scrape targets  
│   └─ alerts.yml          # Reglas de alerta  
├─ hermes/  
│   ├─ skills/             # Skills del agente (auto-gestionadas)  
│   └─ data/               # Estado persistente del agente  
├─ backups/  
│   └─ litellm-YYYYMMDD.db # Backups automáticos  
└─ deploy.log              # Historial de despliegues
```

6. Auto-Curacion y Confiabilidad del Sistema

La version mejorada incorpora mecanismos de auto-curacion que detectan y recuperan automaticamente los contenedores fallidos, reduciendo la necesidad de intervencion manual a traves de 9remote.

6.1 Healthchecks y Docker Auto-Heal

Docker por si solo no reinicia un contenedor cuando su healthcheck falla; solo lo marca como `unhealthy` pero mantiene el proceso en ejecucion. La version mejorada utiliza el contenedor `willfarrell/autoheal` para monitorear continuamente el estado de salud y reiniciar automaticamente cualquier contenedor que entre en estado `unhealthy`.

Configuracion del Servicio Auto-Heal

```
# Extension al docker-compose.yml

services:
  # ... (servicios principales)

  # =====
  # AUTO-HEAL: Supervisor de salud
  # =====
  autoheal:
    image: willfarrell/autoheal:1.2.0
    container_name: autoheal
    restart: unless-stopped
    environment:
      AUTOHEAL_CONTAINER_LABEL: "autoheal.enabled=true"
      AUTOHEAL_INTERVAL: 30      # Verificar cada 30 segundos
      AUTOHEAL_START_PERIOD: 60  # Esperar 60s tras inicio
      AUTOHEAL_DEFAULT_STOP_TIMEOUT: 10
      AUTOHEAL_ONLY_MONITOR_RUNNING: "true"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
    labels:
      autoheal.enabled: "true"
    deploy:
      resources:
        limits:
          cpus: '0.1'
          memory: 32M
```

Para que un servicio sea monitoreado por autoheal, debe incluir la etiqueta `autoheal.enabled=true` y definir un healthcheck en su configuración. Los servicios `litellm`, `hermes` y `whisper` ya incluyen esta etiqueta en su definición del `docker-compose.yml` principal.

Healthcheck Robusto con Kill en Fallo

Como mecanismo adicional, el healthcheck puede configurarse para matar el proceso principal del contenedor cuando falla, lo que activa la política de reinicio de Docker:

```
# Alternativa: healthcheck que fuerza reinicio
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=2 \
    CMD curl -f http://localhost:8080/health || (kill -15 1 && sleep 10 && kill
-9 1)
```

6.2 Servicios Systemd de Supervision

Además del autoheal basado en healthchecks, se configuran servicios systemd en el host para supervisar procesos críticos que no corren dentro de Docker, como Tailscale y el daemon de Docker mismo.

Servicio Systemd para Docker

```
# /etc/systemd/system/docker-watchdog.service

[Unit]
Description=Docker Daemon Watchdog
After=docker.service
Requires=docker.service

[Service]
Type=simple
ExecStart=/usr/local/bin/docker-watchdog.sh
Restart=always
RestartSec=30
StandardOutput=journal
StandardError=journal
SyslogIdentifier=docker-watchdog

[Install]
WantedBy=multi-user.target
```

```
#!/usr/local/bin/docker-watchdog.sh
# docker-watchdog.sh - Script de supervision del daemon Docker

LOG_FILE="/var/log/docker-watchdog.log"

log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" | tee -a "$LOG_FILE"
}

log "Watchdog iniciado"

while true; do
    # Verificar que Docker daemon responde
    if ! docker info > /dev/null 2>&1; then
        log "CRITICO: Docker daemon no responde"
        log "Intentando reiniciar Docker..."
        systemctl restart docker
        sleep 30

        # Verificar si volvio
        if docker info > /dev/null 2>&1; then
            log "Docker recuperado exitosamente"
            # Reiniciar stack
            cd /opt/infra-stack && docker compose up -d
        else
            log "FALLO CRITICO: Docker no pudo recuperarse"
            # Notificar via webhook
            curl -X POST -H 'Content-type: application/json' \
                --data '{"text":"CRITICO: Docker daemon fallido en produccion"}' \
                "$SLACK_WEBHOOK_URL" 2>/dev/null || true
        fi
    fi

    # Verificar contenedores criticos
    for container in litellm-router hermes-agent whisper-stt; do
        status=$(docker inspect --format='{{.State.Status}}' "$container"
        2>/dev/null || echo "missing")
        if [ "$status" != "running" ]; then
            log "AVISO: $container no esta running (estado: $status)"
        fi
    done

    sleep 30
done
```

Activacion del Watchdog

```
chmod +x /usr/local/bin/docker-watchdog.sh
systemctl daemon-reload
systemctl enable docker-watchdog
systemctl start docker-watchdog

# Verificar estado
systemctl status docker-watchdog
journalctl -u docker-watchdog -f
```

7. System Prompt Optimizado para Hermes

El eslabon final para la latencia cero es la inyeccion de la personalidad y las reglas del sistema dentro del prompt central de Hermes. Si el agente genera texto con formato o saludos redundantes, la latencia del TTS aumenta drasticamente debido a los tokens adicionales que deben procesarse.

System Prompt v2.0 (Optimizado para Flash v2.5)

ERES HERMES, un agente autonomo avanzado de gestion operativa.
Tu idioma estricto y unico es el ESPANOL.

REGLAS OPERATIVAS DE EFICIENCIA (OBLIGATORIAS):

1. EXTREMA CONCISION: Responde de forma telegrafica, conversacional y directa. Maximo 25 palabras por respuesta salvo que se solicite detalle explicito.
2. CERO MULETILLAS: Prohibido usar saludos ("Hola", "Claro", "Entendido", "Vale"), despedidas ("Quedo atento", "Un saludo"), o introducciones ("Voy a", "Te comento que"). Ve DIRECTO a la informacion.
3. TEXTO PLANO UNICAMENTE: Prohibido Markdown. No uses viñetas, asteriscos, negritas, codigo, tablas, emojis, ni formatos especiales. Escribe el texto exactamente como debe ser pronunciado por una voz humana natural.
4. CONFIRMACION INMEDIATA: Cuando se solicite confirmar una accion (registrar inventario, procesar notas, ajustar hardware), inicia DIRECTAMENTE con la confirmacion y el resultado final.
Ejemplo: "Listo, registre 3 cervezas y 2 aguas en mesa 5."
5. NUMEROS NATURALES: Expresa numeros como se hablan, no como se escriben.
Correcto: "trescientos cincuenta"
Incorrecto: "350"
6. SIN INFORMACION REDUNDANTE: No repitas lo que el usuario acaba de decir. No expliques tu razonamiento. Solo el resultado.

EJEMPLOS DE RESPUESTAS CORRECTAS:

- "Listo, anote 2 pastas y 1 ensalada para la mesa 3."
- "El inventario de hoy: 45 cervezas, 30 vinos, 12 gaseosas."
- "Reiniciando el router. Conexion restablecida en 30 segundos."

EJEMPLOS DE RESPUESTAS PROHIBIDAS:

- "Hola, con gusto te ayudo con eso. Voy a procesar tu solicitud..."
- "***Confirmacion:** *Se ha registrado exitosamente* 
- "Claro que si! A continuacion el detalle de tu pedido: 1. ... 2. ..."

Razonamiento Detras del Prompt

Cada token generado por el LLM representa aproximadamente 75ms de inferencia adicional en ElevenLabs Flash v2.5. Un saludo de 5 tokens adicionales representan 375ms de latencia extra que el usuario percibe como silencio. Por ello, el prompt prioriza la eliminacion absoluta de todo token no esencial. Las instrucciones se redactan en mayusculas para aumentar su peso atencional en el modelo, y los ejemplos concretos activan el mecanismo de few-shot learning, mejorando significativamente el cumplimiento de las restricciones.

8. Checklist de Despliegue y Referencias

Checklist Pre-Despliegue

Tabla 7: Verificaciones obligatorias antes del primer despliegue

Fase	Verificacion	Comando/Metodo	Criterio
Infraestructura	Tailscale activo	<code>tailscale status</code>	All nodes connected
	Firewall activo	<code>ufw status verbose</code>	Default deny incoming
	Docker hardening	<code>docker info grep -i security</code>	no-new-privileges activo
Red	Grants aplicados	Tailscale admin console	Reglas visibles y activas
	Puertos cerrados	<code>nmap -p 1-65535 localhost</code>	Solo puertos esperados abiertos
	Iptables Docker	<code>iptables -L DOCKER-USER -v</code>	DROP para interfaces publicas
Servicios	Healthchecks	<code>docker ps --format '{{.Names}}\t{{.Status}}'</code>	All (healthy)
	Metricas	Prometheus targets UI	All targets UP
	Logs	<code>docker compose logs --tail 50</code>	Sin errores criticos
Voz	Latencia Flash	Test de voz end-to-end	< 500ms percebida
	Calidad TTS	Escuchar muestras	Naturales, sin artefactos
	Whisper STT	Test de transcripcion	Precision > 95% en espanol
CI/CD	Pipeline verde	GitHub Actions UI	Ultimo run: success
	Rollback test	Simular fallo de deploy	Rollback automatico funciona

Referencias y Fuentes

- [1] ElevenLabs, Inc. (2025). *ElevenLabs WebSocket API Documentation*. Recuperado de <https://elevenlabs.io/docs/api-reference/text-to-speech/v-1-text-to-speech-voice-id-stream-input>
- [2] ElevenLabs, Inc. (2025). *Latency Optimization Guide*. Recuperado de <https://elevenlabs.io/docs/eleven-api/guides/how-to/best-practices/latency-optimization>
- [3] BerriAI. (2025). *LiteLLM Proxy Documentation*. GitHub Repository. <https://github.com/BerriAI/litellm>
- [4] Tailscale Inc. (2025). *Grants: Unified Access Controls*. Tailscale Blog. <https://tailscale.com/blog/june-25-product-update>
- [5] Docker Inc. (2025). *Docker Security Best Practices*. Docker Documentation. <https://docs.docker.com/develop/dev-best-practices/>
- [6] Prometheus Authors. (2025). *Prometheus Monitoring Guide*. <https://prometheus.io/docs/introduction/overview/>
- [7] Google LLC. (2025). *cAdvisor Container Advisor*. <https://github.com/google/cadvisor>
- [8] GitHub, Inc. (2025). *GitHub Actions Documentation*. <https://docs.github.com/en/actions>
- [9] TrueFoundry. (2025). *What is LLM Router?*. <https://www.truefoundry.com/blog/what-is-llm-router>
- [10] OpenAI. (2025). *Whisper: Robust Speech Recognition via Large-Scale Weak Supervision*. <https://github.com/openai/whisper>
- [11] Anthropic PBC. (2025). *Claude API Documentation*. <https://docs.anthropic.com/>
- [12] Linux Foundation. (2025). *systemd.service Documentation*. <https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html>